

File: process-run.py

```
#!/usr/bin/env python

import sys
from optparse import OptionParser
import random

# process switch behavior
SCHED_SWITCH_ON_IO = &#x27;SWITCH_ON_IO&#x27;
SCHED_SWITCH_ON_END = &#x27;SWITCH_ON_END&#x27;

# io finished behavior
IO_RUN_LATER = &#x27;IO_RUN_LATER&#x27;
IO_RUN_IMMEDIATE = &#x27;IO_RUN_IMMEDIATE&#x27;

# process states
STATE_RUNNING = &#x27;RUNNING&#x27;
STATE_READY = &#x27;READY&#x27;
STATE_DONE = &#x27;DONE&#x27;
STATE_WAIT = &#x27;WAITING&#x27;

# members of process structure
PROC_CODE = &#x27;code&#x27;
PROC_PC = &#x27;pc&#x27;
PROC_ID = &#x27;pid&#x27;
PROC_STATE = &#x27;proc_state&#x27;

# things a process can do
DO_COMPUTE = &#x27;cpu&#x27;
DO_IO = &#x27;io-start&#x27;

class scheduler:
    def __init__(self, process_switch_behavior, io_done_behavior, io_length):
        # keep set of instructions for each of the processes
        self.proc_info = {}
        self.process_switch_behavior = process_switch_behavior
        self.io_done_behavior = io_done_behavior
        self.io_length = io_length
        return

    def new_process(self):
        proc_id = len(self.proc_info)
        self.proc_info[proc_id] = {}
        self.proc_info[proc_id][PROC_PC] = 0
        self.proc_info[proc_id][PROC_ID] = proc_id
        self.proc_info[proc_id][PROC_CODE] = []
        self.proc_info[proc_id][PROC_STATE] = STATE_READY
        return proc_id

    def load_file(self, progfile):
        fd = open(progfile)
        proc_id = self.new_process()

        for line in fd:
            tmp = line.split()
            if len(tmp) == 0:
                continue
            opcode = tmp[0]
            if opcode == &#x27;compute&#x27;;:
                assert(len(tmp) == 2)
                for i in range(int(tmp[1])):
                    self.proc_info[proc_id][PROC_CODE].append(DO_COMPUTE)
            elif opcode == &#x27;io&#x27;;:
                assert(len(tmp) == 1)
                self.proc_info[proc_id][PROC_CODE].append(DO_IO)
        fd.close()
        return

    def load(self, program_description):
```

```

proc_id = self.new_process()
tmp = program_description.split('&#x27;:&#x27;')
if len(tmp) != 2:
    print &#x27;Bad description (%s): Must be number &lt;x:y>&#x27; % program_description
    print &#x27; where X is the number of instructions&#x27;
    print &#x27; and Y is the percent change that an instruction is CPU not IO&#x27;
    exit(1)

num_instructions, chance_cpu = int(tmp[0]), float(tmp[1])/100.0
for i in range(num_instructions):
    if random.random() &lt; chance_cpu:
        self.proc_info[proc_id][PROC_CODE].append(DO_COMPUTE)
    else:
        self.proc_info[proc_id][PROC_CODE].append(DO_IO)
return

def move_to_ready(self, expected, pid=-1):
    if pid == -1:
        pid = self.curr_proc
    assert(self.proc_info[pid][PROC_STATE] == expected)
    self.proc_info[pid][PROC_STATE] = STATE_READY
    return

def move_to_wait(self, expected):
    assert(self.proc_info[self.curr_proc][PROC_STATE] == expected)
    self.proc_info[self.curr_proc][PROC_STATE] = STATE_WAIT
    return

def move_to_running(self, expected):
    assert(self.proc_info[self.curr_proc][PROC_STATE] == expected)
    self.proc_info[self.curr_proc][PROC_STATE] = STATE_RUNNING
    return

def move_to_done(self, expected):
    assert(self.proc_info[self.curr_proc][PROC_STATE] == expected)
    self.proc_info[self.curr_proc][PROC_STATE] = STATE_DONE
    return

def next_proc(self, pid=-1):
    if pid != -1:
        self.curr_proc = pid
        self.move_to_running(STATE_READY)
        return
    for pid in range(self.curr_proc + 1, len(self.proc_info)):
        if self.proc_info[pid][PROC_STATE] == STATE_READY:
            self.curr_proc = pid
            self.move_to_running(STATE_READY)
            return
    for pid in range(0, self.curr_proc + 1):
        if self.proc_info[pid][PROC_STATE] == STATE_READY:
            self.curr_proc = pid
            self.move_to_running(STATE_READY)
            return
    return

def get_num_processes(self):
    return len(self.proc_info)

def get_num_instructions(self, pid):
    return len(self.proc_info[pid][PROC_CODE])

def get_instruction(self, pid, index):
    return self.proc_info[pid][PROC_CODE][index]

def get_num_active(self):
    num_active = 0
    for pid in range(len(self.proc_info)):
        if self.proc_info[pid][PROC_STATE] != STATE_DONE:
            num_active += 1
    return num_active

```

```

def get_num_runnable(self):
    num_active = 0
    for pid in range(len(self.proc_info)):
        if self.proc_info[pid][PROC_STATE] == STATE_READY or \
            self.proc_info[pid][PROC_STATE] == STATE_RUNNING:
            num_active += 1
    return num_active

def get_ios_in_flight(self, current_time):
    num_in_flight = 0
    for pid in range(len(self.proc_info)):
        for t in self.io_finish_times[pid]:
            if t > current_time:
                num_in_flight += 1
    return num_in_flight

def check_for_switch(self):
    return

def space(self, num_columns):
    for i in range(num_columns):
        print '&#x27;%10s&#x27; % &#x27; &#x27;,'

def check_if_done(self):
    if len(self.proc_info[self.curr_proc][PROC_CODE]) == 0:
        if self.proc_info[self.curr_proc][PROC_STATE] == STATE_RUNNING:
            self.move_to_done(STATE_RUNNING)
            self.next_proc()
    return

def run(self):
    clock_tick = 0

    if len(self.proc_info) == 0:
        return

    # track outstanding IOs, per process
    self.io_finish_times = {}
    for pid in range(len(self.proc_info)):
        self.io_finish_times[pid] = []

    # make first one active
    self.curr_proc = 0
    self.move_to_running(STATE_READY)

    # OUTPUT: headers for each column
    print '&#x27;%s&#x27; % &#x27;Time&#x27;,'
    for pid in range(len(self.proc_info)):
        print '&#x27;%10s&#x27; % (&#x27;PID:%2d&#x27; % (pid)),
    print '&#x27;%10s&#x27; % &#x27;CPU&#x27;,'
    print '&#x27;%10s&#x27; % &#x27;IOs&#x27;,'
    print '&#x27;&#x27;,'

    # init statistics
    io_busy = 0
    cpu_busy = 0

    while self.get_num_active() > 0:
        clock_tick += 1

        # check for io finish
        io_done = False
        for pid in range(len(self.proc_info)):
            if clock_tick in self.io_finish_times[pid]:
                io_done = True
                self.move_to_ready(STATE_WAIT, pid)
                if self.io_done_behavior == IO_RUN_IMMEDIATE:
                    # IO_RUN_IMMEDIATE
                    if self.curr_proc != pid:
                        if self.proc_info[self.curr_proc][PROC_STATE] == STATE_RUNNING:

```

```

        self.move_to_ready(STATE_RUNNING)
        self.next_proc(pid)
    else:
        # IO_RUN_LATER
        if self.process_switch_behavior == SCHED_SWITCH_ON_END and self.get_num_runnable() > 0:
            # this means the process that issued the io should be run
            self.next_proc(pid)
        if self.get_num_runnable() == 1:
            # this is the only thing to run: so run it
            self.next_proc(pid)
        self.check_if_done()

    # if current proc is RUNNING and has an instruction, execute it
    instruction_to_execute = None
    if self.proc_info[self.curr_proc][PROC_STATE] == STATE_RUNNING and \
        len(self.proc_info[self.curr_proc][PROC_CODE]) > 0:
        instruction_to_execute = self.proc_info[self.curr_proc][PROC_CODE].pop(0)
        cpu_busy += 1

    # OUTPUT: print what everyone is up to
    if io_done:
        print '%3d' % clock_tick,
    else:
        print '%3d' % clock_tick,
    for pid in range(len(self.proc_info)):
        if pid == self.curr_proc and instruction_to_execute != None:
            print '%10s' % ('%s;RUN:' % instruction_to_execute),
        else:
            print '%10s' % (self.proc_info[pid][PROC_STATE]),
    if instruction_to_execute == None:
        print '%10s' % ('');
    else:
        print '%10s' % 1,
    num_outstanding = self.get_ios_in_flight(clock_tick)
    if num_outstanding > 0:
        print '%10s' % str(num_outstanding),
        io_busy += 1
    else:
        print '%10s' % ('');
    print '%s' % ('');

    # if this is an IO instruction, switch to waiting state
    # and add an io completion in the future
    if instruction_to_execute == DO_IO:
        self.move_to_wait(STATE_RUNNING)
        self.io_finish_times[self.curr_proc].append(clock_tick + self.io_length)
        if self.process_switch_behavior == SCHED_SWITCH_ON_IO:
            self.next_proc()

    # ENDCASE: check if currently running thing is out of instructions
    self.check_if_done()
    return (cpu_busy, io_busy, clock_tick)

```

```

#
# PARSE ARGUMENTS
#

```

```

parser = OptionParser()
parser.add_option('-s', '--seed', default=0, help='the random seed', action='store_true')
parser.add_option('-l', '--processlist', default='', help='a comma-separated list of processes to run, in the form X1:Y1,X2:Y2,... where X is process id and Y is priority', action='store', type='string', dest='process_list')
parser.add_option('-L', '--iolength', default=5, help='how long an IO takes', action='store', type='int', dest='io_length')
parser.add_option('-S', '--switch', default='SWITCH_ON_IO', help='when to switch between processes: SWITCH_ON_IO, SWITCH_ON_END', action='store', type='string', dest='process_switch_behavior')
parser.add_option('-I', '--iodone', default='IO_RUN_LATER', help='type of behavior when IO ends: IO_RUN_LATER, IO_RUN_IMMEDIATE', action='store', type='string', dest='io_done_behavior')
parser.add_option('-c', '--compute', help='compute answers for me', action='store_true', default=False)

```

```

parser.add_option('--p', &#x27;--printstats&#x27;, help=&#x27;print statistics at end; only useful with
(options, args) = parser.parse_args()

random.seed(options.seed)

assert(options.process_switch_behavior == SCHED_SWITCH_ON_IO or \
options.process_switch_behavior == SCHED_SWITCH_ON_END)
assert(options.io_done_behavior == IO_RUN_IMMEDIATE or \
options.io_done_behavior == IO_RUN_LATER)

s = scheduler(options.process_switch_behavior, options.io_done_behavior, options.io_length)

# example process description (10:100,10:100)
for p in options.process_list.split(&#x27;;&#x27;):
    s.load(p)

if options.solve == False:
    print &#x27;Produce a trace of what would happen when you run these processes:&#x27;
    for pid in range(s.get_num_processes()):
        print &#x27;Process %d&#x27; % pid
        for inst in range(s.get_num_instructions(pid)):
            print &#x27; %s&#x27; % s.get_instruction(pid, inst)
        print &#x27;&#x27;
    print &#x27;Important behaviors:&#x27;
    print &#x27; System will switch when&#x27;,
    if options.process_switch_behavior == SCHED_SWITCH_ON_IO:
        print &#x27;the current process is FINISHED or ISSUES AN IO&#x27;
    else:
        print &#x27;the current process is FINISHED&#x27;
    print &#x27; After IOs, the process issuing the IO will&#x27;,
    if options.io_done_behavior == IO_RUN_IMMEDIATE:
        print &#x27;run IMMEDIATELY&#x27;
    else:
        print &#x27;run LATER (when it is its turn)&#x27;
    print &#x27;&#x27;
    exit(0)

(cpu_busy, io_busy, clock_tick) = s.run()

if options.print_stats:
    print &#x27;&#x27;
    print &#x27;Stats: Total Time %d&#x27; % clock_tick
    print &#x27;Stats: CPU Busy %d (%.2f%)&#x27; % (cpu_busy, 100.0 * float(cpu_busy)/clock_tick)
    print &#x27;Stats: IO Busy %d (%.2f%)&#x27; % (io_busy, 100.0 * float(io_busy)/clock_tick)
    print &#x27;&#x27;

```